INTERNSHIP PROJECT REPORT

Diya

An unattended meditation kiosk

A visitor is identified by their own phone, guided through a camera-observed session, and handed a report. Then the machine resets itself.

[Your Name] · [Institution]

[Month Year – Month Year]

Guide: [Mentor Name]

SCROLL

THE BRIEF

A visitor walks up. Nobody is there to help them.

A museum kiosk has to identify a stranger, run a camera-guided meditation session, hand them a personal report, and reset itself — unattended, all day.

01

No staff

Nobody is present to check anyone in, or to restart anything that stops.

02

No keyboard

A shared touchscreen that strangers type on needs cleaning between visitors.

03

No scanner

Dedicated reader hardware is cost, procurement and one more thing that breaks.

04

Never closed

An unattended machine that crashes and stays down is worth nothing at all.

SCOPE

Two teams, one thin seam

HARDWARE TEAM · IITH

Cameras, servos, CV

Depth and thermal capture, posture and gaze analysis, and the external meditation-app that produces the report PDF.

MINE

Everything around it

The kiosk application, the identification system, the backend and web pages, packaging and deployment — and the contract between the two sides.

> **The seam is two primitives, not a protocol.** I launch one shell script and read one PDF off disk. They changed their internals all summer without ever breaking my app.

THE HARD PART

How do you identify someone with no scanner?

Every obvious answer costs hardware — a QR reader, a card reader, or a keyboard that strangers share.

DESIGN EVOLUTION

Three attempts

01

Data in the QR

The printed code carried the visitor's details. Fully offline — but it still needs a reader, and it puts personal data on paper.

`DIYA1:<base64 json>`

02

QR holds an id

The code became a lookup key and the kiosk fetched details over the API. Better privacy; the scanner problem remained.

`GET /api/visitors/:id`

03 · SHIPPED

Invert it

The kiosk shows the QR and the visitor's phone scans it. The phone is the scanner, the camera and the keyboard. Zero hardware.

`POST /api/sessions`

THE DECISION THAT MATTERED

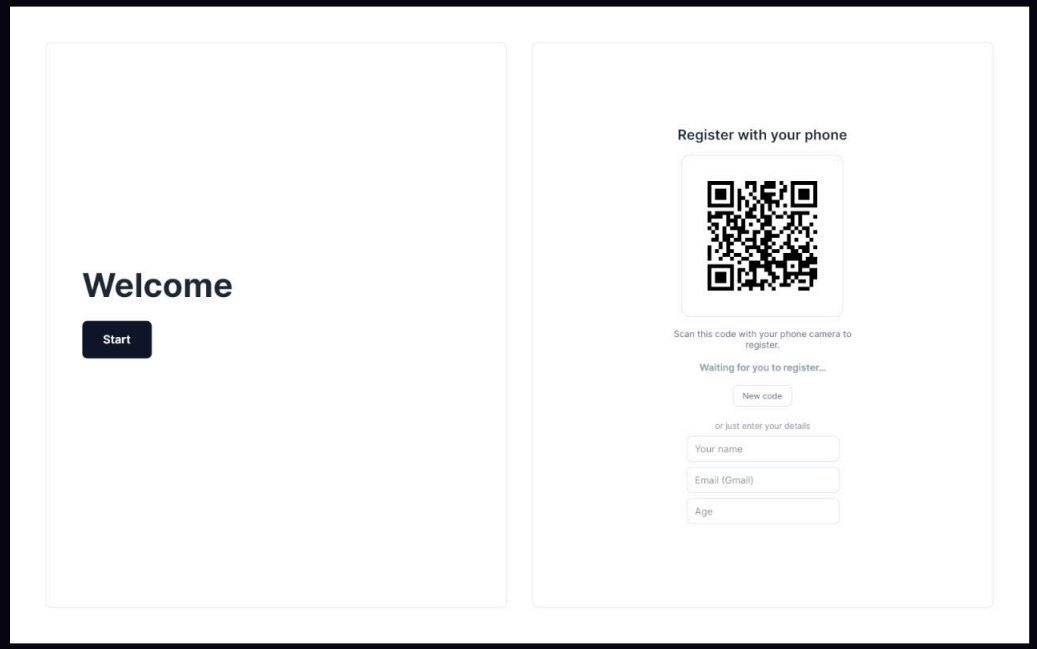
The phone is the scanner

The kiosk asks the backend for a session, renders the token as a QR, and polls. The visitor's own phone does the reading.

`POST /api/sessions` `GET /api/sessions/:token`

`POST /api/claim`

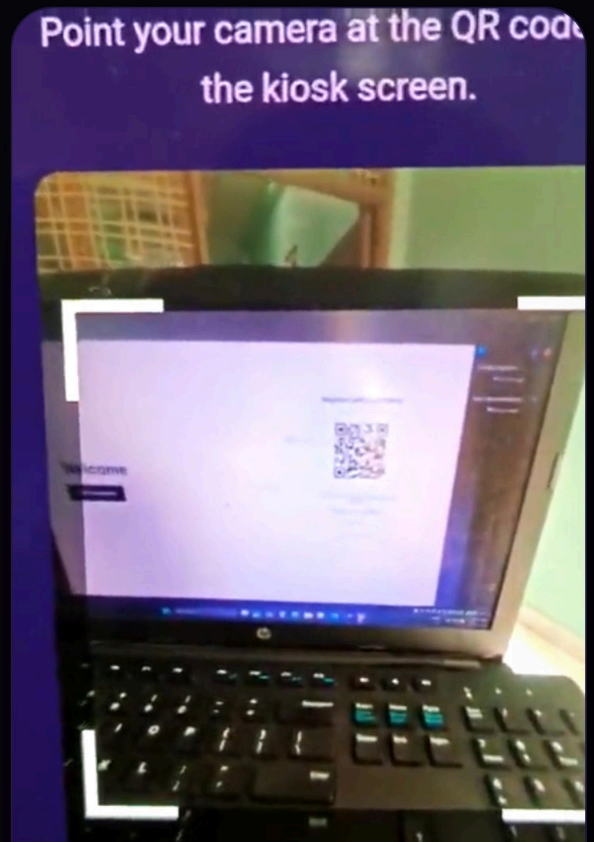
> A constraint produced a simpler design than a budget would have.

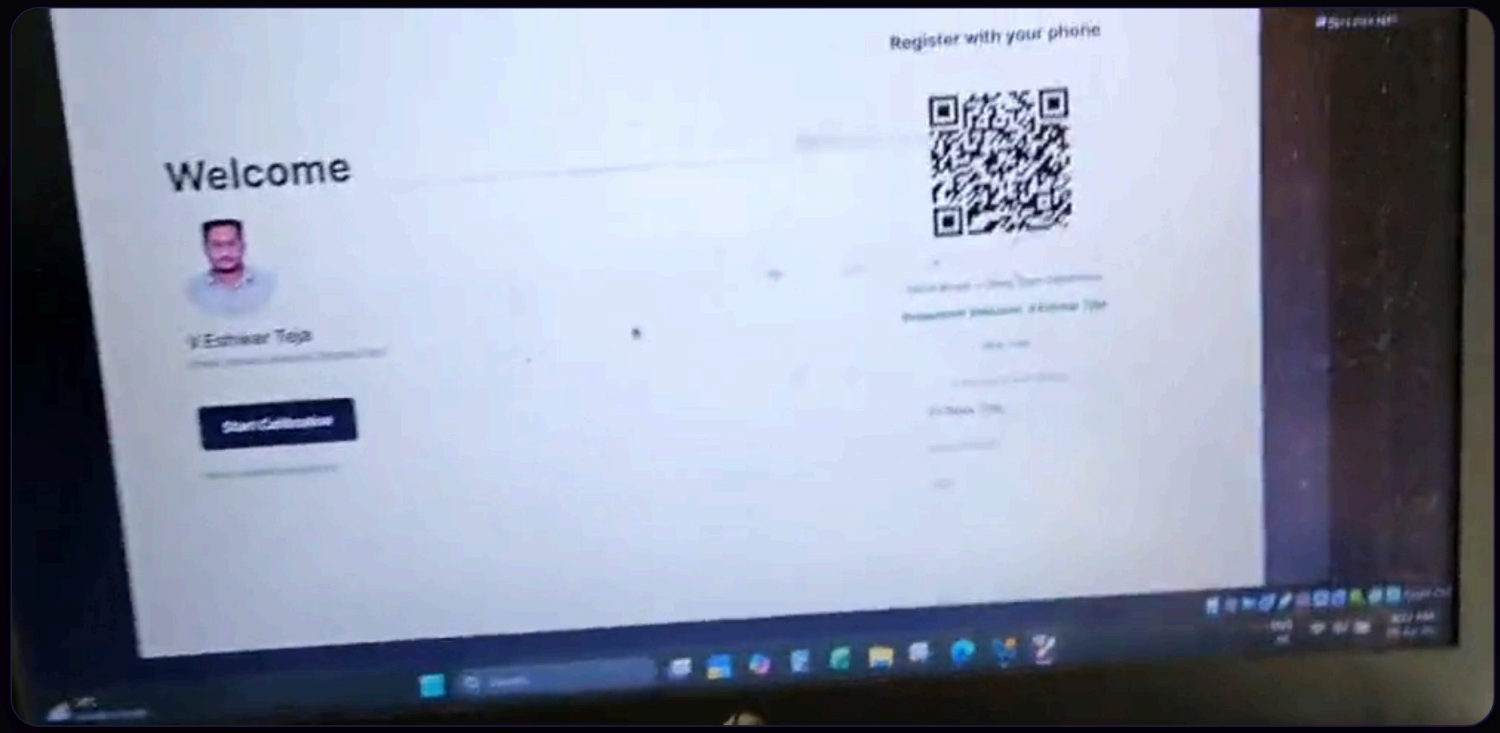


SEE IT WORK

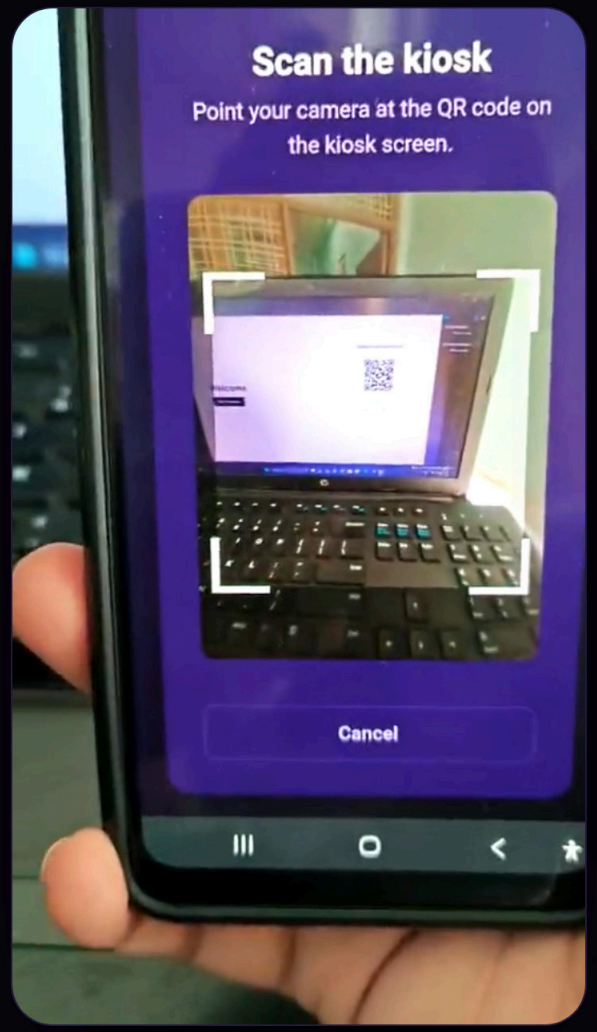
The scan, on real hardware

Filmed during a live run. The phone opens its private link, points at the kiosk, and the kiosk takes over on its own.

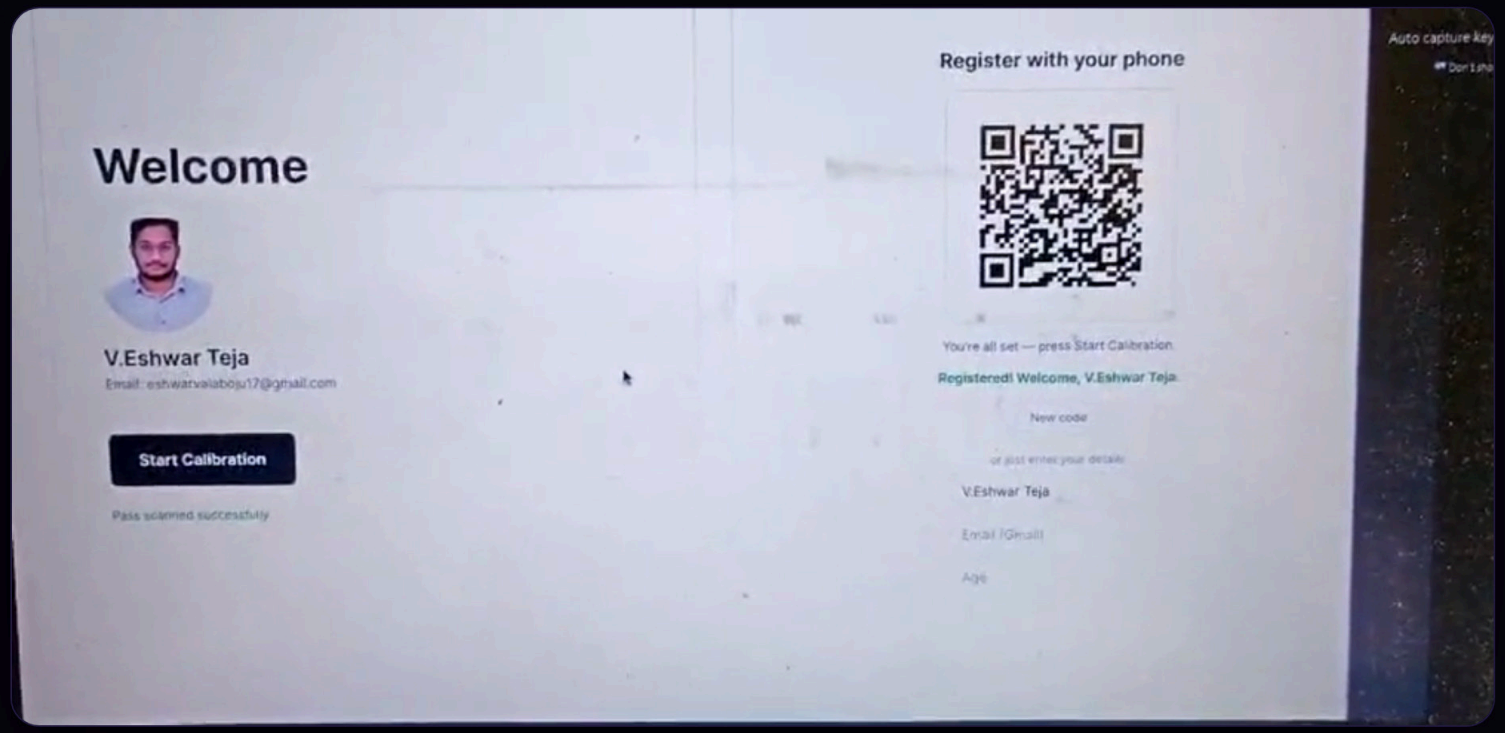




The kiosk responds. Name and roster photo resolve from the claim, and the session begins without anyone pressing anything.



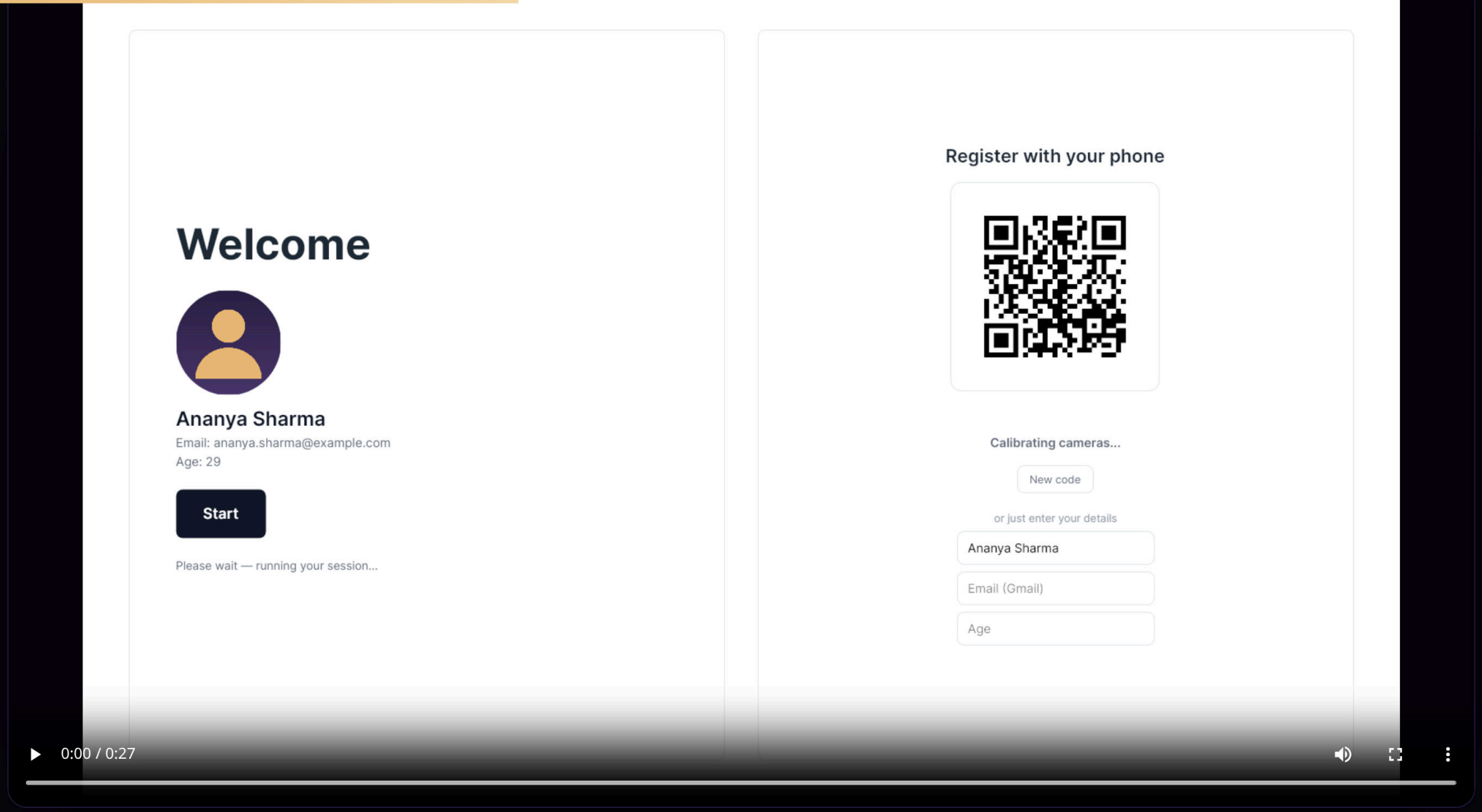
The phone reads the kiosk. No app, no typing, no scanner hardware — the camera the visitor already owns does the work.



The kiosk responds. Name and roster photo resolve from the claim, and the session begins without anyone pressing anything.

> Two seconds of polling is all that connects them. The phone never talks to the kiosk directly — both only talk to the backend.

Full walkthrough



The complete run, unedited — roster already loaded, through to the authenticated kiosk.

END TO END

Four stages, one API

STAGE 01

Identify

An admin roster gives each person a private link. They open it on their phone and scan the kiosk.

`GET /api/people/:token`

STAGE 02

Claim

The backend links that identity to the kiosk's open session. The polling kiosk advances by itself.

`POST /api/claim`

STAGE 03

Session

The kiosk launches the CV pipeline and blocks. This boundary is deliberately not an API.

`bash scripts/run1.sh`

STAGE 04

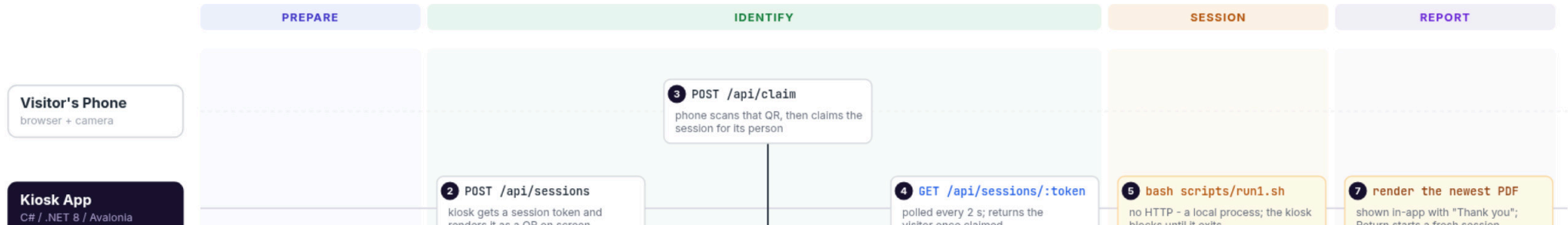
Report

The newest PDF written after the session began is rendered in-app, then the kiosk resets.

`PDFium → images`

How the kiosk and the backend talk: one REST API, four phases

Time flows left to right. The hardware team's app is the one boundary that is deliberately *not* an API.



person a private link. They open it on their phone and scan the kiosk.

`GET /api/people/:token`

to the kiosk's open session. The polling kiosk advances by itself.

`POST /api/claim`

pipeline and blocks. This boundary is deliberately not an API.

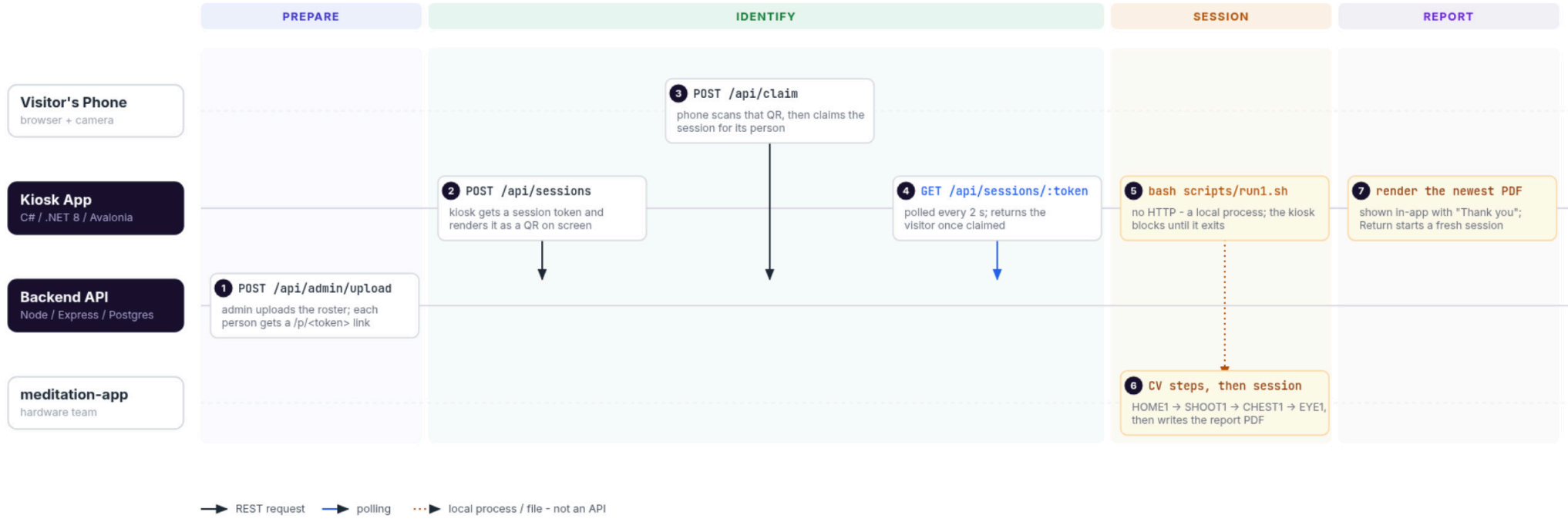
`bash scripts/run1.sh`

the session began is rendered in-app, then the kiosk resets.

`PDFium → images`

How the kiosk and the backend talk: one REST API, four phases

Time flows left to right. The hardware team's app is the one boundary that is deliberately *not* an API.

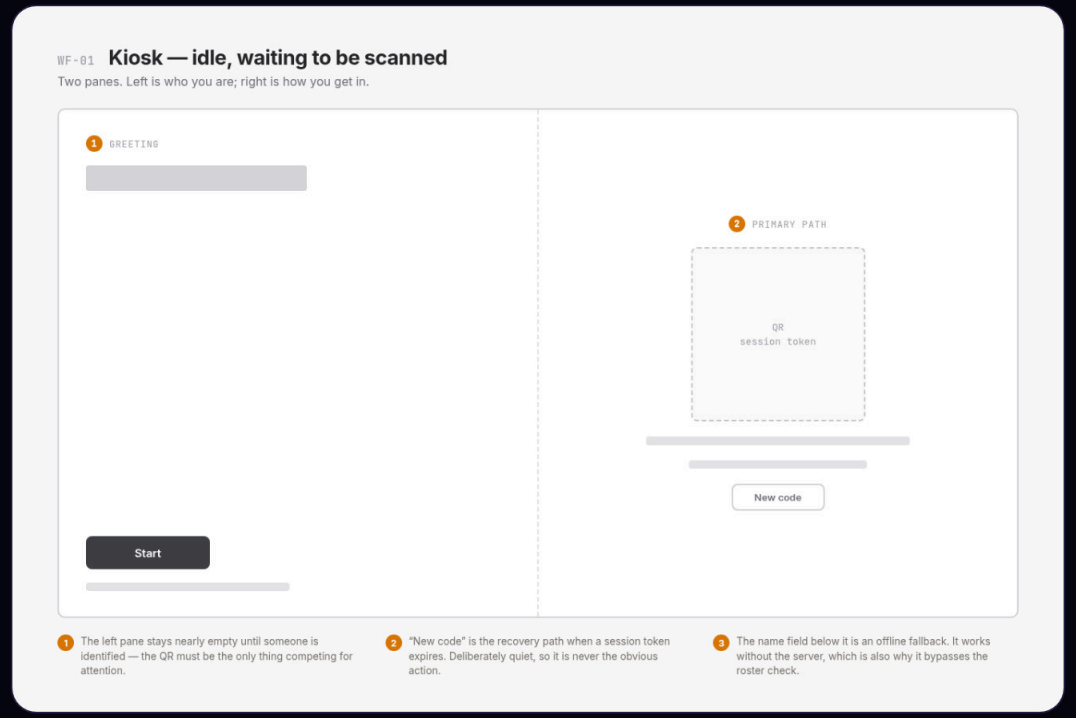


Kiosk to Backend is the only network boundary — plain REST over HTTPS. **Kiosk to meditation-app** is a local process and a file on disk, so there is no protocol to break.

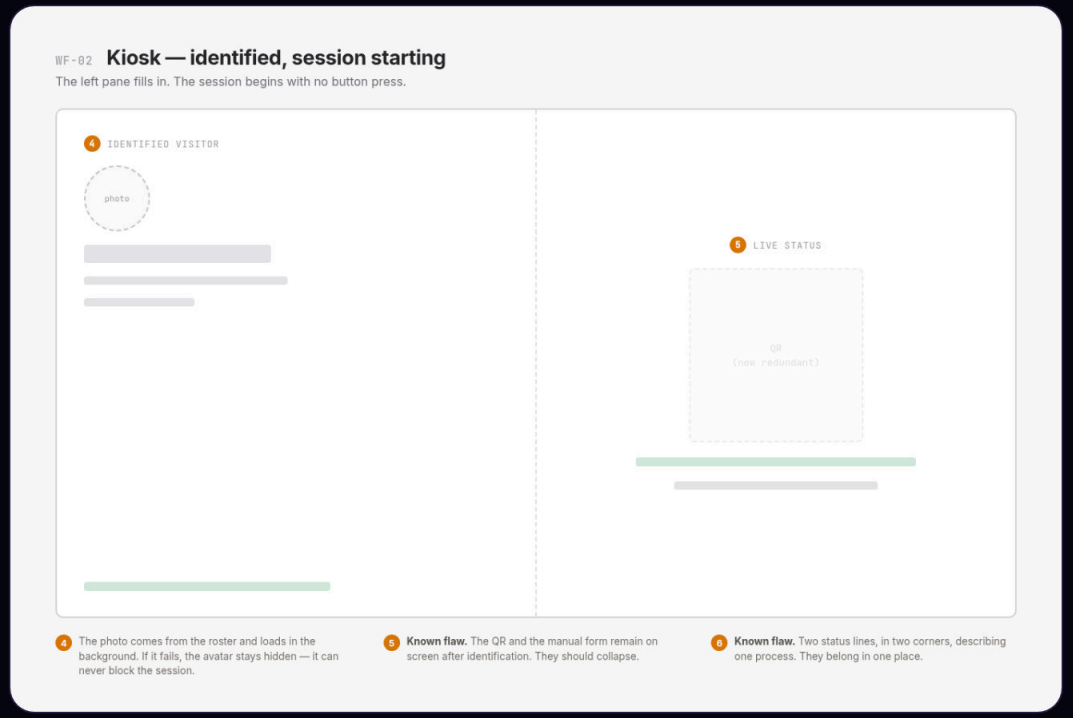
DESIGN PROCESS

Wireframes, and what they got wrong

Annotated before building. Two of the notes are flaws I only saw once it was running — they are marked as such rather than quietly fixed in the drawing.

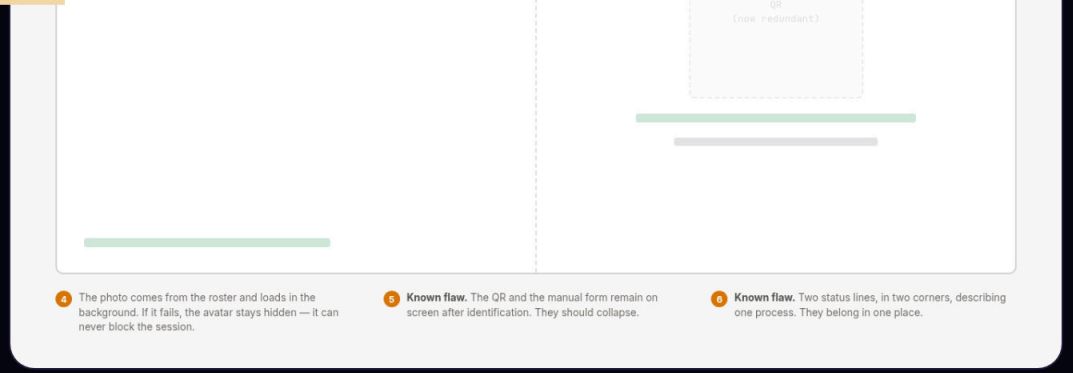


Idle. The QR owns the screen; everything else is deliberately quiet.

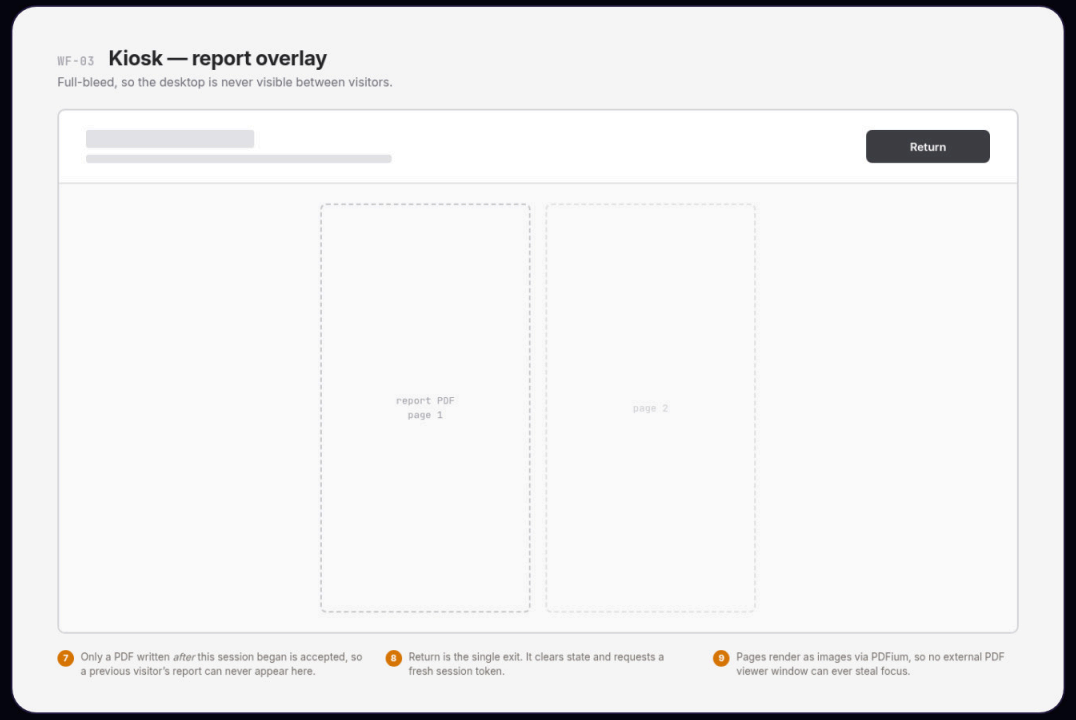


Identified. Notes 5 and 6 are real defects — stale controls and split status.

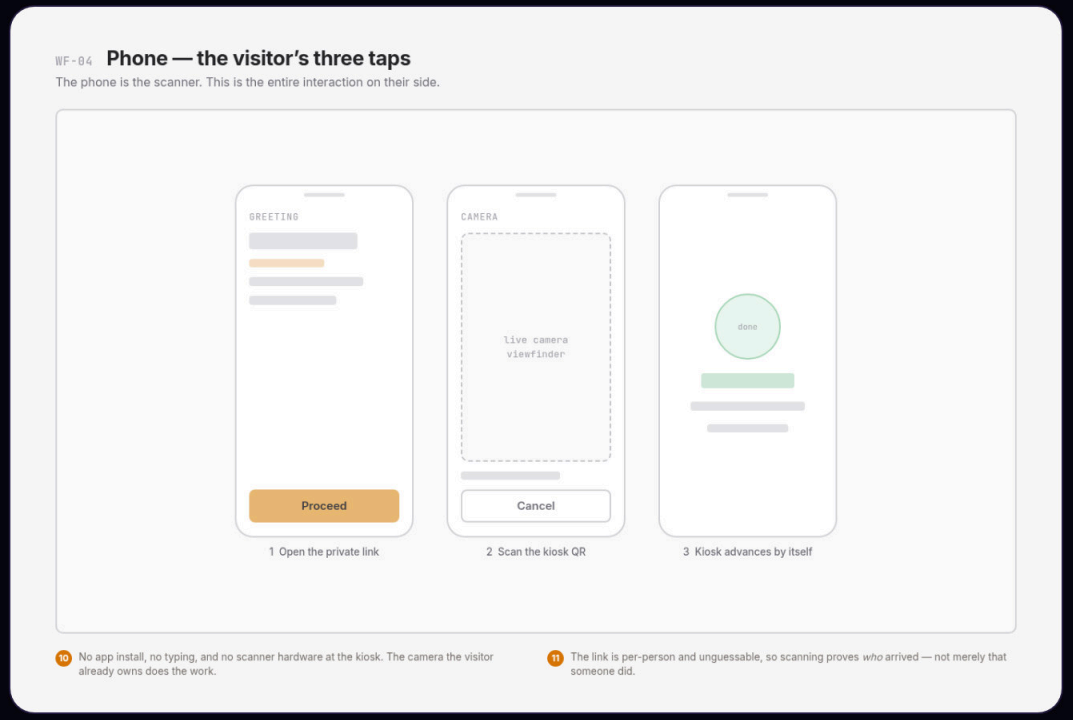
Idle. The QR owns the screen; everything else is deliberately quiet.



Identified. Notes 5 and 6 are real defects — stale controls and split status.



Report overlay, full-bleed so the desktop never shows between visitors.

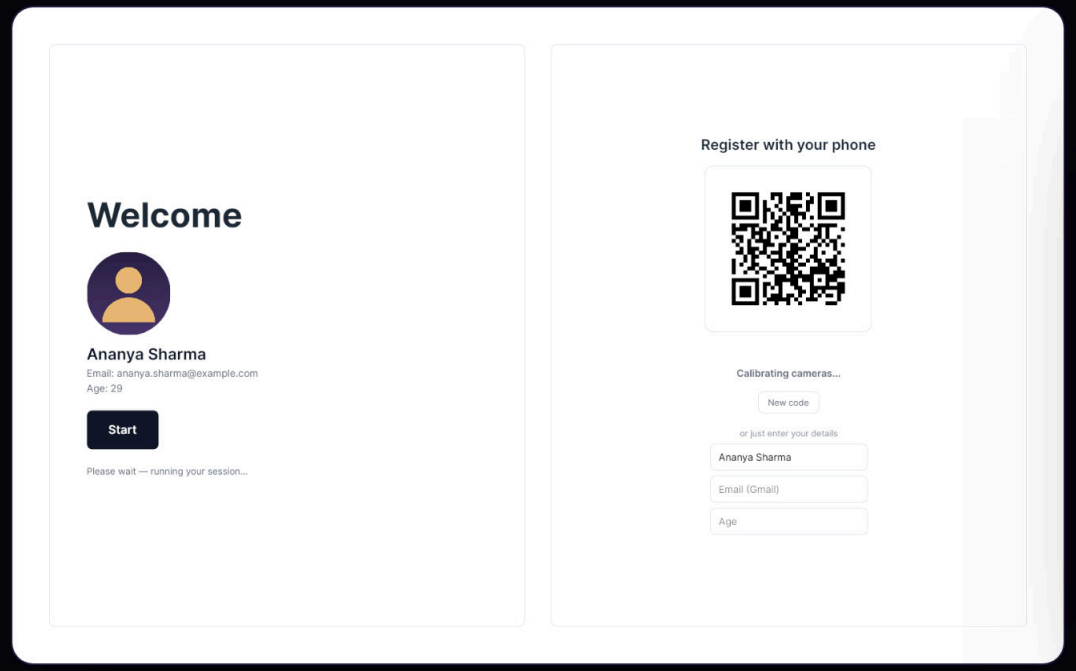


The phone side: three taps, no app install, no typing.

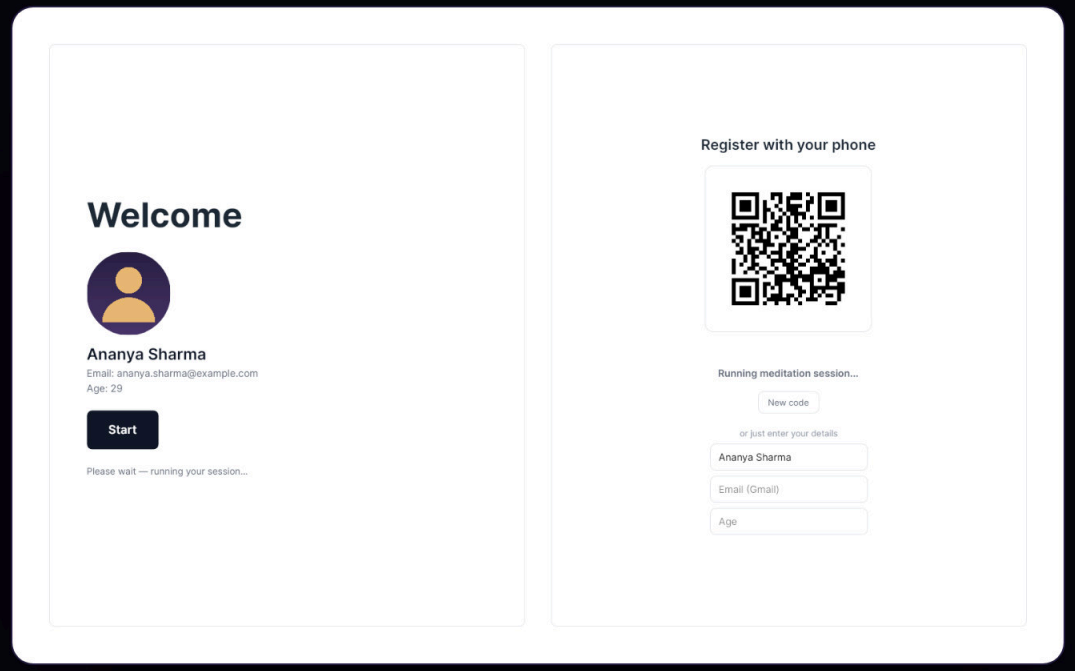
RUNNING SYSTEM

Every screen, for real

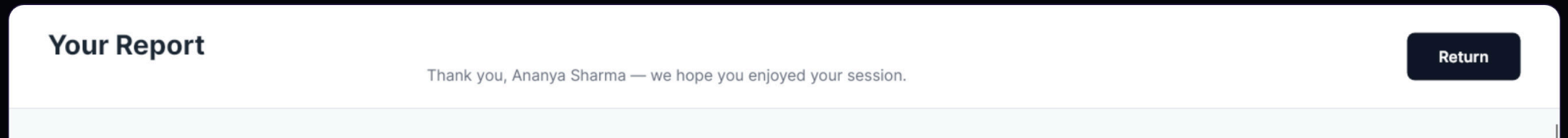
Captured from the running application and the live web pages — not mockups.

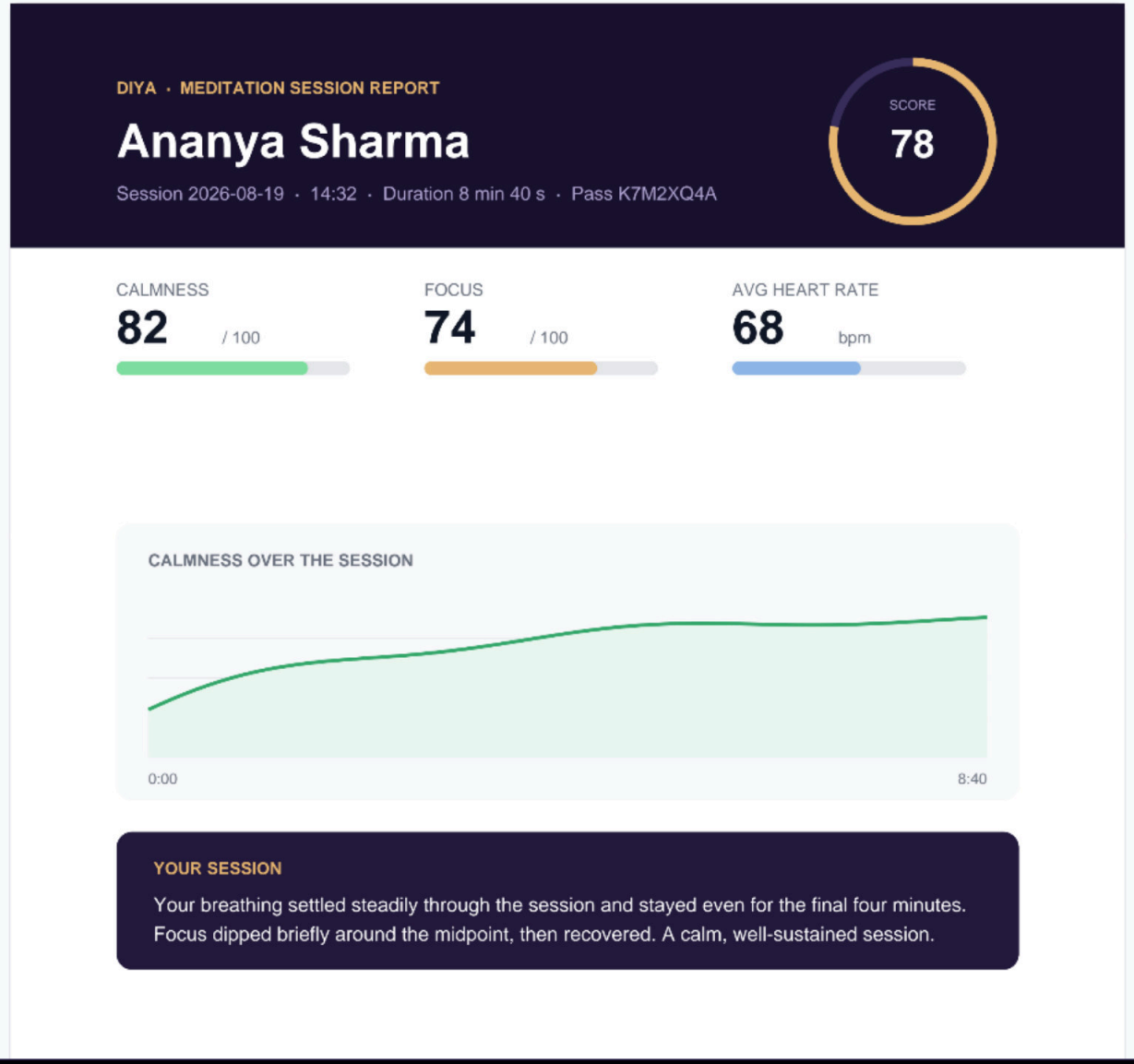


Identified. Name, email and roster photo arrive from the claim; the session starts with no button press.



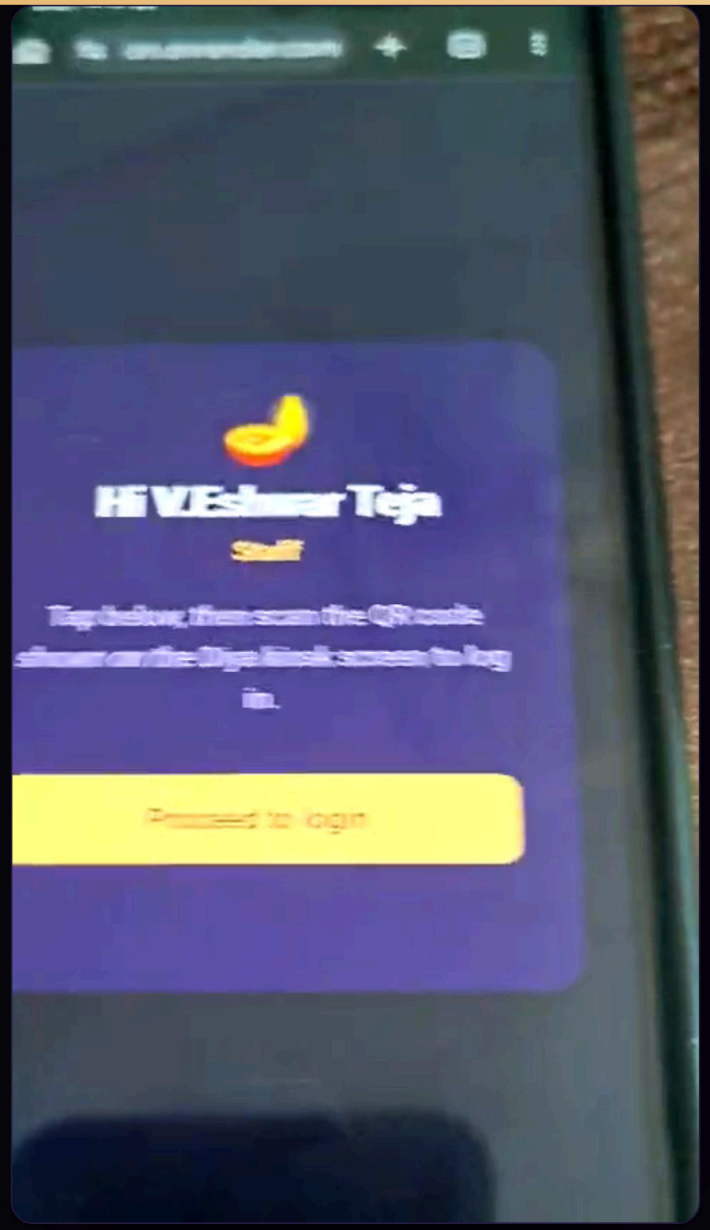
Session running. The live status line is the pipeline's own stdout, stripped of colour codes.



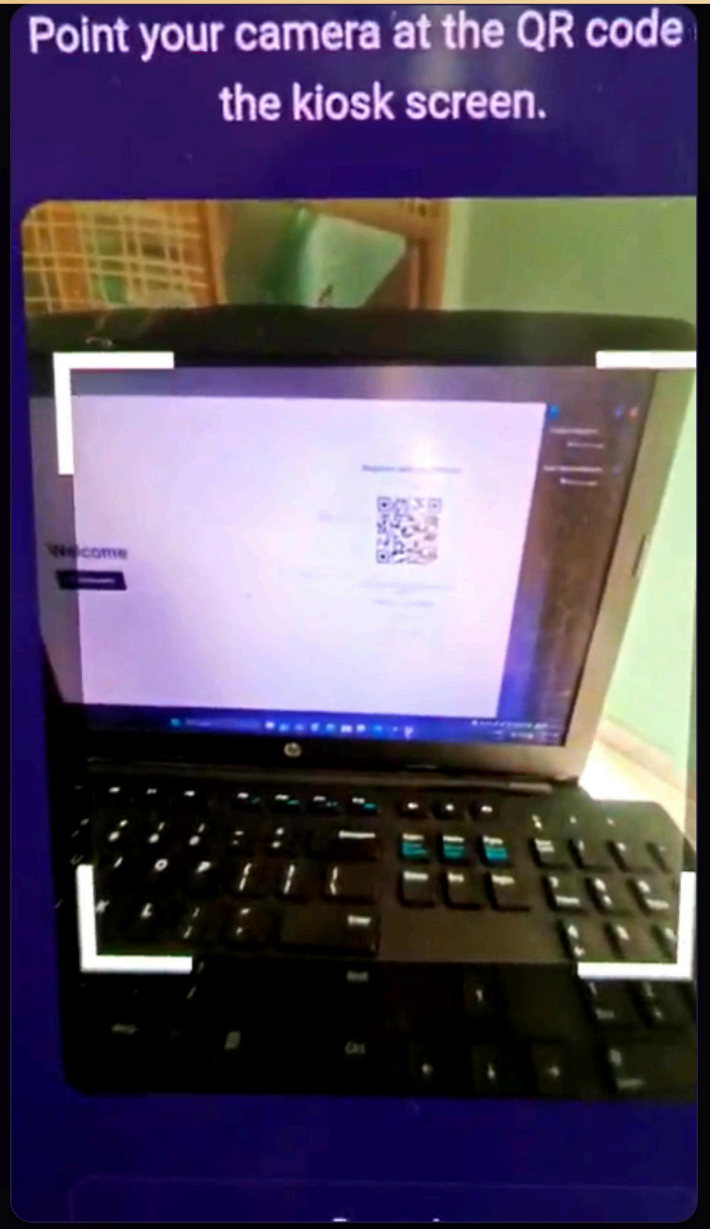


Report. The PDF is decoded by PDFium and rendered inside the app — no external viewer, and the desktop is never exposed.

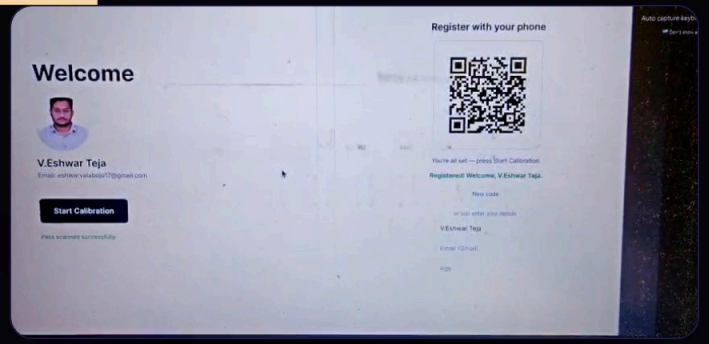
From the live run



The private link greets the person by name, then opens the camera.

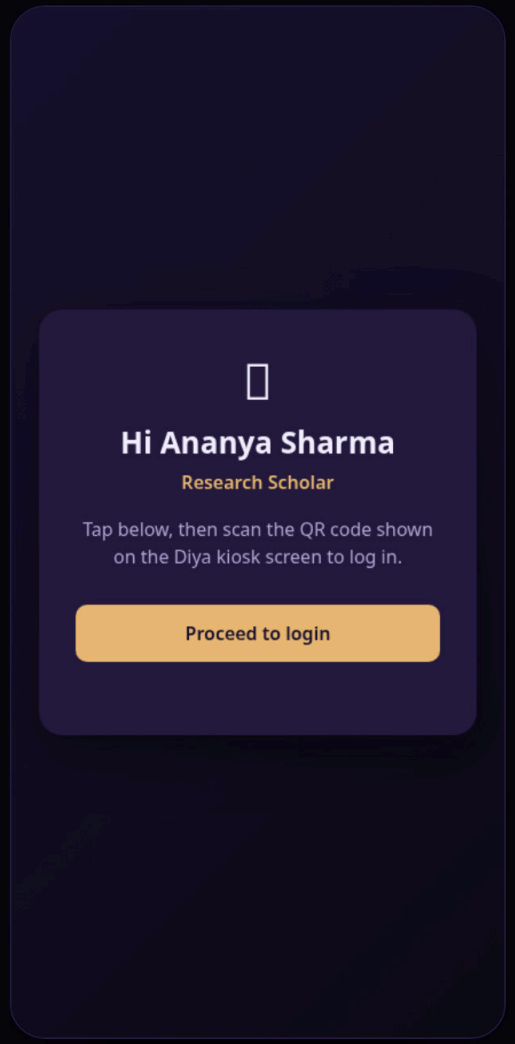


The viewfinder framing the QR on the kiosk screen.

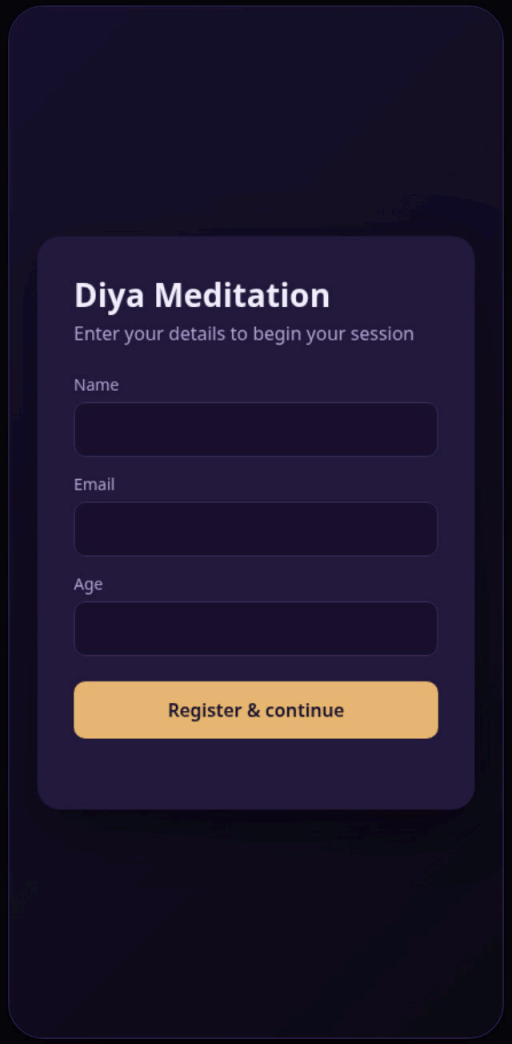


The kiosk after the claim — name, photo, and the session ready to run.

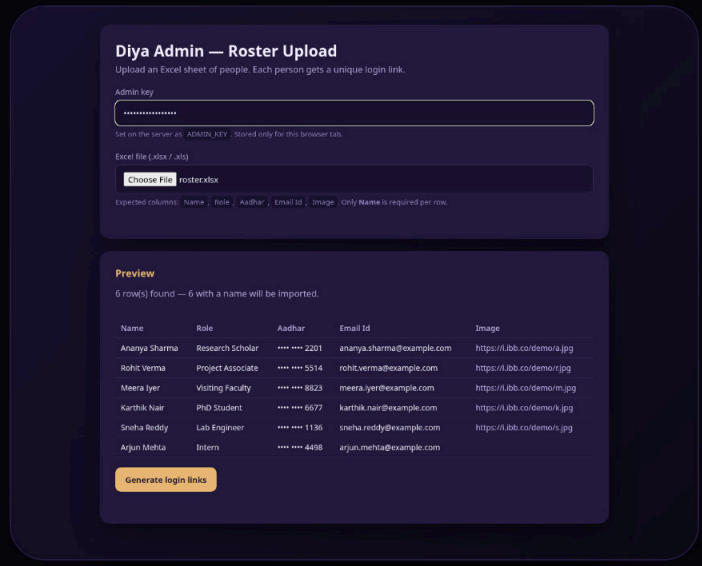
The web side



Private link. Greets the person by name. Returns name and role only — never Aadhaar or email.



Self-registration. The session-aware fallback when someone is not on the roster.



Admin roster. XLSX parsed in the browser, Aadhaar masked in the preview.

Three parts I built

KIOSK APP

C# · .NET 8 · Avalonia

The only .NET UI framework that officially supports Linux. Ships as a self-contained single-file .deb for two architectures.

BACKEND

Node · Express · Postgres

Sessions, visitors and the admin roster, serving the web pages from the same origin so the phone camera gets HTTPS.

WEB

Three static pages

Phone registration, admin roster upload with in-browser XLSX parsing, and the per-person login scanner.

nia · QRCoder · PDFium · SkiaSharp · Avalonia · QRCoder

ystemd · Postgres · SheetJS · html5-qrcode · systemd · Pos

8
source files in the kiosk app

9
API endpoints

3
Postgres tables

2
architectures packaged

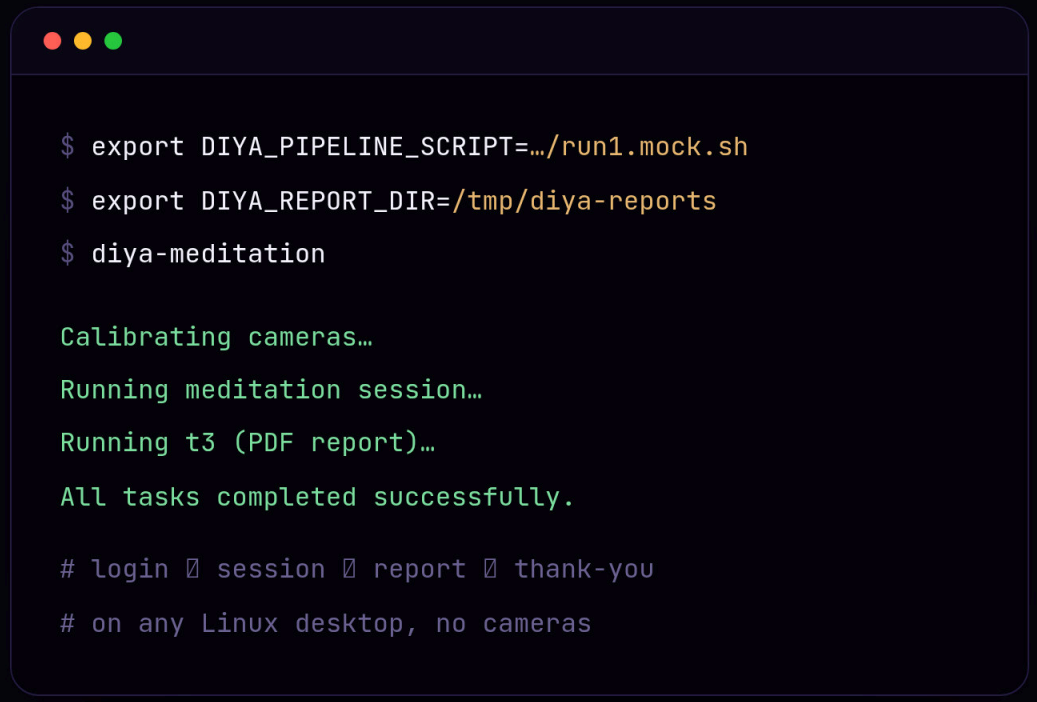
TESTING

Building it without the hardware

For most of the internship there were no cameras and no meditation-app. The flow still had to be testable end to end.

> Two environment variables swap the pipeline for a mock. **No test code inside the app**, nothing extra in the shipped package — and those same variables are the real production override.

It also surfaced a real bug: “show the newest PDF” would quietly hand a visitor their predecessor’s report. Now only a file newer than the session start is accepted.



SHIPPING

One command each

BUILD

A single package

Self-contained .deb for amd64 and arm64. The museum machine needs nothing pre-installed — not even .NET.

```
./deploy/build-deb.sh 1.6.0 amd64
```

LOCK DOWN

A kiosk, not an app

A systemd user service restarts it on crash, autologin brings it up on boot, and the GNOME escape shortcuts are disabled.

```
Restart=always
```

DEPLOY

Backend in one file

A blueprint provisions the web service and Postgres together, then tracks main on every merge.

```
render.yaml
```

> Plus documentation written for whoever comes next: an overview, a full command reference, an FAQ, and a handoff note that records the **dead ends** so nobody repeats them.

WHAT COMES NEXT

Where I handed it over

The full path works today: roster upload, phone login, automatic session, report on screen, reset. Four things are queued for whoever continues.

01

Personalise the report

Pass the visitor into the pipeline so the PDF is theirs rather than generic.

02

Bound the pipeline

A timeout and a failure screen, so a dead camera cannot strand the kiosk.

03

Consent screen

Cameras observe visitors and identity data is stored. It needs asking first.

04

Real CV hardware

The interfaces are in place; the devices arrive after my internship ends.

Someone can walk up, log in with their phone, and leave with their report.

The vision hardware is the remaining milestone. Everything I could not finish is written down.

Thank you — questions?

Screenshots are captured from the running application. The report PDF shown is representative — the real one is produced by the hardware team's meditation-app.

Animated components from React Bits.